

Paper

“`latex`”

1 Summary

Bài báo *Can It Reach the Generator? Investigating the Survival of Prompt-Injection Attacks in Realistic RAG Settings* nghiên cứu độ hiệu quả thực tế của các *prompt-injection attacks* trong hệ thống *Retrieval-Augmented Generation* (RAG), đặc biệt trong bối cảnh *Generative Engine Optimization* (GEO). Mục tiêu của các attack này là chỉnh sửa nội dung tài liệu hoặc mô tả sản phẩm sao cho sản phẩm mục tiêu được mô hình ngôn ngữ ưu tiên đề xuất trong câu trả lời cuối cùng.

Vấn đề chính mà bài báo đặt ra là nhiều nghiên cứu GEO trước đây đánh giá attack trong một setting quá đơn giản. Cụ thể, các nghiên cứu đó thường giả định rằng tài liệu bị tấn công đã luôn nằm trong context đầu vào của generator. Nói cách khác, attacked document được đưa trực tiếp vào LLM generator, rồi đo xem nó có được recommend hay không. Bài báo gọi setting này là *Frozen Context* (FC). Tuy nhiên, trong hệ thống RAG thực tế, một tài liệu muốn ảnh hưởng đến generator phải đi qua nhiều tầng trước đó, gồm retrieval và reranking. Nếu attack làm thay đổi nội dung tài liệu quá mạnh, tài liệu đó có thể không còn được retriever lấy về, hoặc bị reranker đẩy xuống dưới top- k , và như vậy không thể tới generator.

Do đó, bài báo đặt câu hỏi trung tâm: một prompt-injection attack có thực sự *survive* qua toàn bộ pipeline RAG hay không? Để trả lời, tác giả đánh giá attack trong pipeline ba tầng thực tế hơn:

Retriever $\rightarrow$ LLM Reranker $\rightarrow$ LLM Generator

Trong pipeline này, attack không được gắn trực tiếp vào context của generator. Thay vào đó, attacked document được đưa lại vào corpus, hệ thống re-index, sau đó chạy retrieval, reranking và generation từ đầu. Đây là khác biệt quan trọng giữa evaluation thực tế và evaluation kiểu frozen-context.

Về mặt hình thức, paper ký hiệu corpus là:

$$C = \{d_1, d_2, \dots, d_N\}$$

với query là q . Attacker chọn một target document $d^\star$ và chỉnh sửa nó thành:

$$\tilde{d}^\star = d^\star \oplus \delta$$

trong đó δ là adversarial edit. Corpus sau attack là:

$$\tilde{C} = (C \setminus \{d^\star\}) \cup \{\tilde{d}^\star\}$$

Điểm quan trọng là hệ thống phải chạy lại retrieval trên $\tilde{C}$, chứ không được giả định $\tilde{d}^\star$ đã nằm sẵn trong input context.

Ở tầng đầu tiên, retriever r_N nhận query q và corpus C , sau đó tạo ranking:

$$C_q = r_N(q, C) = (c_1, c_2, \dots, c_N)$$

Retriever có thể là sparse retriever như BM25 hoặc dense retriever như embedding-based retrieval. Trong thực nghiệm, paper dùng cả BM25 và *BAAI/bge-large-en-v1.5* để kiểm tra tác động của loại retriever.

Ở tầng thứ hai, reranker ρ_n nhận top- n documents từ retriever và sắp xếp lại chúng:

$$M_q = \rho_n(q, C_q^{(n)})$$

Paper sử dụng LLM reranker theo chiến lược *listwise RankGPT*. Reranker đóng vai trò quan trọng vì dù attacked document được retriever lấy về, nó vẫn cần được reranker xếp đủ cao để lọt vào generator input.

Ở tầng thứ ba, generator g_k nhận top- k documents sau reranking và sinh câu trả lời:

$$a \sim p_{LLM}(\cdot \mid \pi_{gen}(q, M_q^{(k)}))$$

Trong bài toán product recommendation, generator được yêu cầu tạo ra top- m product recommendations. Paper thiết lập $n = 10$, $k = 5$, và $m = 3$, tức reranker xử lý top-10 candidates, generator nhận top-5, và output gồm top-3 sản phẩm được recommend.

Để đo attack end-to-end, bài báo định nghĩa ba chỉ số stage-specific. Chỉ số thứ nhất là *Retrieval Survival*:

$$S_r(\delta) = \mathbb{1}[\tilde{d}^\star \in \tilde{C}_q^{(n)}]$$

Chỉ số này đo liệu attacked document có sống sót qua tầng retrieval và lọt vào top- n candidates hay không. Nếu $S_r = 0$, attack thất bại ngay từ đầu.

Chỉ số thứ hai là *Reranking Exposure*:

$$E_\rho(\delta) = \mathbb{1}[\tilde{d}^\star \in \tilde{M}_q^{(k)}]$$

Chỉ số này đo liệu attacked document có được reranker đưa vào top- k documents được

truyền cho generator hay không.

Chỉ số thứ ba là *Generation Success*:

$$S_g(\delta) = \mathbb{1}[\text{title}(d^*) \subseteq \tilde{a}]$$

Chỉ số này đo liệu tên sản phẩm mục tiêu có xuất hiện trong output recommendation cuối cùng hay không. Một attack end-to-end thành công phải thỏa:

$$S_g(\delta) = 1 \Rightarrow E_\rho(\delta) = 1 \Rightarrow S_r(\delta) = 1$$

Điều này cho thấy generator success chỉ có ý nghĩa thực tế nếu attacked document thật sự đi qua được retrieval và reranking.

Thực nghiệm được tiến hành trên *Amazon ESCI product-search corpus*, một dataset có query-product relevance labels gồm bốn mức: *Exact*, *Substitute*, *Complement*, và *Irrelevant*. Paper giữ lại các query có ít nhất 40 annotated products để phản ánh tình huống e-commerce search thực tế hơn. Sau đó, tác giả chọn 200 queries cho mỗi retriever, gồm BM25 và dense retriever.

Paper đánh giá bảy attack methods thuộc ba nhóm chính. Nhóm thứ nhất là *Instruction Override Attack* (IOA), tức chèn instruction trực tiếp để yêu cầu model ưu tiên target product. Nhóm thứ hai là *LLM-driven prompt optimization*, gồm CORE-Review, CORE-Reason và TAP; nhóm này dùng LLM phụ để sinh hoặc tinh chỉnh prompt-injection content. Nhóm thứ ba là *gradient-based attacks*, gồm RAF, SRP và STS; nhóm này tối ưu token bằng gradient signal để đẩy target item lên cao.

Một điểm thiết kế quan trọng là paper attack các document ở hai vị trí trong ranking ban đầu: rank 10 và rank 6. Rank 10 là document cuối cùng mà reranker nhìn thấy trong top- n . Rank 6 là document đầu tiên bị loại khỏi generator input vì generator chỉ nhận top-5. Cách chọn này giúp kiểm tra liệu attack có thể đẩy một document từ vùng rìa vào vùng được generator thấy hay không.

Kết quả đầu tiên cho thấy reranker có ảnh hưởng hai mặt. Trong frozen-context setting, khi retrieval bị bỏ qua, reranker có thể giúp một số attacks bằng cách đẩy attacked document lên cao hơn. IOA, CORE-review, CORE-reason và TAP được reranker promote khá mạnh, đạt AvgRank khoảng 3.25 và tăng $S_g@3$ đáng kể. Tuy nhiên, khi generator chỉ nhận top-5 sau reranking, cơ chế truncation lại làm giảm success của một số attack, đặc biệt là TAP, vì attacked document có thể nằm ở rank 6–10 và bị loại khỏi generator input.

Kết quả thứ hai cho thấy retrieval là một bộ lọc quan trọng. Khi chuyển từ Frozen Context sang End-to-End, retrieval survival không còn là 100%. Với attack ở rank 10, trung bình chỉ khoảng 79.9% attacked documents còn sống sót qua retrieval. Nói cách khác, khoảng một phần năm attacks thất bại trước khi đến reranker. IOA bị ảnh hưởng nặng nhất vì suffix instruction làm loãng nội dung lexical/semantic liên quan đến query,

khiến document khó được retriever lấy về. Trong khi đó, LLM-driven attacks sống sót tốt hơn vì chúng thường giữ hoặc tăng query-relevant content.

Kết quả thứ ba là end-to-end attack effectiveness phụ thuộc rất mạnh vào loại attack. Trong setting thực tế E2E, chỉ nhóm LLM-driven prompt optimization còn hiệu quả rõ rệt. CORE-reason đạt $S_g@3$ khoảng 53.5%, CORE-review khoảng 46.7%, và TAP khoảng 40.2%. Ngược lại, gradient-based attacks như SRP, RAF và STS gần như sụp đổ, với success rate dưới 2%. IOA cũng giảm mạnh, còn khoảng 14.7%, dù nó từng rất mạnh trong frozen-context setting.

Một phát hiện đáng chú ý là CORE-reason thậm chí được retrieval khuếch đại trong một số trường hợp. Vì CORE-reason sinh nội dung có chứa query nhiều lần và vẫn giữ tính liên quan semantic, retriever có thể xếp attacked document cao hơn. Điều này cho thấy retrieval không chỉ là bộ lọc chống attack; nó cũng có thể trở thành amplifier nếu attack được thiết kế theo hướng query-aligned.

Bài báo cũng đánh giá defense bằng prompt-injection guards. Các guard off-the-shelf gồm Llama Guard, Qwen Guard và Prompt Guard không generalize tốt trên mọi attack. Prompt Guard phát hiện tốt IOA và STS nhưng thất bại với CORE-review, CORE-reason và TAP. Llama Guard và Qwen Guard cũng có hiệu năng không ổn định, đặc biệt khi class imbalance giống deployment thực tế.

Tuy nhiên, khi finetune một lightweight Prompt Guard trên một small attack dataset, paper đạt kết quả rất cao. Biến thể *PG-FT* đạt F1 trung bình khoảng 96.9% trong balanced setting và khoảng 81.2% trong pipeline setting. Điều này cho thấy các attacks còn sống sót end-to-end có schema tương đối dễ học, nghĩa là threat landscape thực tế hẹp hơn so với những gì frozen-context evaluation gợi ý.

Một phân tích kỹ thuật khác là *stage-level position preference*. Paper cho thấy reranker và generator có bias vị trí khác nhau. Reranker có xu hướng ưu tiên attack đặt ở vị trí muộn hơn trong input, trong khi generator lại ưu tiên attack đặt ở vị trí sớm hơn. Điều này tạo ra một mâu thuẫn trong pipeline: vị trí tối ưu để qua reranker chưa chắc là vị trí tối ưu để ảnh hưởng generator. Đây là một insight chỉ xuất hiện khi phân tích pipeline nhiều tầng, còn single-stage evaluation sẽ bỏ qua.

Từ các kết quả trên, bài báo rút ra ba nguyên tắc thiết kế attack có thể sống sót end-to-end. Thứ nhất là *query-aligned content*: attack phải giữ hoặc tăng khả năng retrievable của document. Thứ hai là *joint effectiveness across pipeline stages*: attack không được chỉ tối ưu cho generator mà phải hiệu quả qua cả retrieval, reranking và generation. Thứ ba là *robustness to position bias*: attack phải hoạt động ổn định ở nhiều vị trí input khác nhau vì từng tầng có position bias riêng.

2 Conclusion

Hướng đi đã làm của bài báo là đánh giá lại các prompt-injection attacks trong GEO/RAG recommendation bằng một pipeline end-to-end thực tế hơn. Thay vì giả định attacked document luôn nằm trong context của generator, bài báo buộc document phải đi qua retriever, LLM reranker và LLM generator. Cách đánh giá này cho thấy nhiều kết quả attack success trong các nghiên cứu trước đã bị phóng đại do bỏ qua các tầng upstream.

Đóng góp kỹ thuật quan trọng nhất của bài báo là framework đánh giá attack theo ba stage-specific indicators: *Retrieval Survival*, *Reranking Exposure*, và *Generation Success*. Ba chỉ số này tách rõ attack thất bại ở đâu trong pipeline. Nếu attack không qua retrieval, đó là failure ở S_r ; nếu qua retrieval nhưng không lọt top- k sau reranking, đó là failure ở E_p ; nếu vào generator input nhưng không xuất hiện trong output, đó là failure ở S_g .

Kết quả thực nghiệm cho thấy frozen-context protocol đánh giá quá cao mức độ nguy hiểm của GEO prompt injection. Khi dùng pipeline end-to-end, gradient-based attacks gần như collapse dưới 2% success rate, instruction override giảm mạnh, và chỉ các LLM-driven attacks như CORE-review, CORE-reason và TAP còn hiệu quả đáng kể. Điều này cho thấy threat landscape thực tế hẹp hơn, nhưng cũng tinh vi hơn: attack hiệu quả phải vừa giữ query relevance, vừa thuyết phục reranker, vừa ảnh hưởng generator.

Bài báo cũng chỉ ra rằng retrieval và reranking không chỉ là bước kỹ thuật phụ, mà là thành phần quyết định trong độ an toàn của RAG system. Retrieval có thể lọc attack nếu adversarial edit làm mất relevance, nhưng cũng có thể khuếch đại attack nếu nội dung attack được tối ưu theo query. Reranker có thể promote attack nhưng top- k truncation có thể chặn nó trước khi tới generator. Vì vậy, đánh giá bảo mật RAG cần xem toàn bộ pipeline thay vì chỉ xem generator.

Hướng kế thừa từ bài báo có thể gồm nhiều nhánh. Thứ nhất, có thể mở rộng đánh giá sang các RAG pipeline phức tạp hơn có query rewriting, personalized retrieval, conversational memory, tool use hoặc post-generation moderation. Thứ hai, có thể kiểm tra trên các domain rủi ro cao như finance, healthcare hoặc legal search, nơi prompt-injection không chỉ ảnh hưởng visibility sản phẩm mà còn có thể gây hậu quả nghiêm trọng hơn. Thứ ba, có thể nghiên cứu defense theo từng stage, ví dụ guard ở indexing stage, retrieval-stage anomaly detection, reranker-level injection detection, hoặc generator-side citation verification.

Ngoài ra, paper này có thể kế thừa trực tiếp cho hướng nghiên cứu về *AI commercial recommendation* và *GEO/AEO*. Các nghiên cứu trước thường quan tâm làm sao để brand/product được LLM recommend, nhưng paper này cho thấy một câu hỏi quan trọng hơn là: nội dung tối ưu hóa đó có thể sống sót qua toàn bộ pipeline RAG hay không? Với các hệ recommender dựa trên RAG, visibility không chỉ là vấn đề viết prompt-injection mạnh ở tầng generator, mà là bài toán đi qua một chuỗi lọc gồm retrieval, reranking và generation. Do đó, cả offensive evaluation và defensive evaluation đều nên chuyển từ

generator-only sang end-to-end pipeline evaluation. ““